UNIVERSITY OF TEXAS AT AUSTIN
MCCOMBS SCHOOL OF BUSINESS
GENERATIVE AI 285N · MSBA 285

MediQuery

A Retrieval-Augmented Generation System
for Medicare Policy Question Answering

Mohar Chaudhuri
Shruthi Chembu Kuppuswamy
Sanchal Nachappa
Janani Vakkanti
Ryota Yokoyama

GitHub Repository: MediQuery-RAG

March 15, 2026

Abstract

Navigating Medicare coverage policy is a persistent challenge for healthcare providers, administrators, and patients alike. Thousands of federal and state-level policy documents govern what procedures and treatments are reimbursable under Medicare, yet these documents are fragmented across government repositories, written in dense regulatory language, and frequently updated. This project presents **MediQuery**, a Retrieval-Augmented Generation (RAG) system that answers natural language questions about Medicare coverage by grounding responses in real, official policy documents — specifically, National Coverage Determinations (NCDs) and Local Coverage Determinations (LCDs) sourced directly from the Centers for Medicare & Medicaid Services (CMS).

We describe the full system pipeline: raw document acquisition from CMS, preprocessing and structured chunking of 1,162 policy documents into 3,377 semantically coherent chunks, semantic embedding via `BAAI/bge-base-en-v1.5`, two-stage retrieval with cross-encoder reranking via `BAAI/bge-reranker-v2-m3`, and grounded answer generation using Mistral-7B-Instruct. The system is evaluated across three configurations — a baseline LLM, RAG, and RAG with query rewriting — measuring retrieval recall, answer faithfulness, citation accuracy, and hallucination rate. Our results demonstrate that RAG substantially reduces hallucination and improves citation accuracy compared to an ungrounded language model baseline.

Contents

1 Introduction **4**

2 Literature Review **4**

2.1 Retrieval-Augmented Generation . . . 4

2.2 RAG in Healthcare . . . 5

2.3 Embedding Models and Rerankers . . . 5

2.4 Hallucination and Faithfulness . . . 5

3 Project Scope and Objectives **6**

3.1 Problem Statement . . . 6

3.2 Objectives . . . 6

3.3 Scope Boundaries . . . 6

4 Data Collection and Preprocessing **6**

4.1 Overview of Data Sources . . . 6

4.2 National Coverage Determinations (NCDs) . . . 7

4.2.1 What NCDs Are . . . 7

4.2.2 How We Downloaded NCD Data . . . 7

4.3 Local Coverage Determinations (LCDs) . . . 8

4.3.1 What LCDs Are . . . 8

4.3.2 Examples of LCDs in Our Corpus . . . 8

4.3.3 How We Downloaded LCD Data . . . 8

4.4 Medicare Benefit Policy Manual (Publication 100-02) . . . 9

4.4.1 What the Benefit Policy Manual Is . . . 9

4.4.2 How We Downloaded the Benefit Policy Manual . . . 10

4.5 Medicare Claims Processing Manual (Publication 100-04) . . . 10

4.5.1 What the Claims Processing Manual Is . . . 10

4.5.2 How We Downloaded the Claims Processing Manual . . . 10

4.6 Final Corpus Summary . . . 10

5 Step 1: Document Chunking and Corpus Construction **11**

5.1 Why Chunking Is Necessary . . . 11

5.2 Chunking Strategy and Parameters . . . 11

5.3 Metadata Tagging Design . . . 11

5.4 Implementation . . . 12

5.5 Corpus Statistics . . . 13

6 Step 2: Embeddings and Vector Index **14**

6.1 Embedding Model Selection . . . 14

6.2 Encoding the Corpus . . . 14

6.3 FAISS Index Construction . . . 15

6.4 Index Statistics . . . 16

6.5 Retrieval Sanity Test . . . 16

7 Step 3: Retrieval Pipeline and Reranking **18**

7.1 Two-Stage Retrieve-Then-Rerank Architecture . . . 18
7.2 Stage 1: Dense Retrieval . . . 18
7.3 State-Aware Filtering . . . 18
7.4 Stage 2: Cross-Encoder Reranking . . . 19
7.5 Bi-Encoder vs Cross-Encoder Models . . . 19
7.6 Retrieve and Rerank Implementation . . . 19
7.7 Before and After Reranking Comparison . . . 20
7.8 Fine-Tuning the Bi-Encoder . . . 20
7.8.1 Training Data Construction . . . 20
7.8.2 Hard Negative Mining . . . 21
7.8.3 Train / Validation / Test Split . . . 21
7.8.4 Loss Function and Hyperparameters . . . 21
7.8.5 Re-Encoding and Index Rebuild . . . 22
7.9 Fine-Tuning the Cross-Encoder Reranker . . . 22
7.9.1 Hard Negative Re-Mining . . . 22
7.9.2 Loss Function and Hyperparameters . . . 22
7.10 Retrieval Improvement After Fine-Tuning . . . 23
7.10.1 Impact of Reranking on the Fine-Tuned Pipeline . . . 23
7.10.2 Performance by Document Type . . . 24

8 Step 4: LLM Generation and Citation **25**
8.1 Generator Choice and Role in the Pipeline . . . 25
8.2 Prompt Design for Grounded Generation . . . 25
8.3 Structured Output and Citation Handling . . . 26
8.4 Example Grounded Outputs . . . 26
8.5 Baseline vs. Grounded Generation . . . 27
8.6 Prompt Engineering Iterations . . . 27

9 System Architecture **28**

10 Evaluation **29**
10.1 Test Set Construction and Evaluation Philosophy . . . 29
10.2 Synthetic Ground Truth Dataset Construction . . . 29
10.2.1 Phase 1: Retrieval Benchmark (Query and Gold Chunk Pairs) . . . 29
10.2.2 Phase 2: Final Answer Evaluation Dataset (Question and Answer Pairs) . . . 29
10.3 Evaluation Metrics . . . 30
10.4 Results . . . 30

11 App Development **31**
11.1 Application Overview . . . 31
11.2 Core Features . . . 31
11.3 User Workflow . . . 31
11.4 Screenshots and Interface Notes . . . 32

12 Conclusion **34**

1. Introduction

Medicare is the United States federal health insurance program covering approximately 65 million Americans aged 65 and older or living with qualifying disabilities. Determining what Medicare covers for a given patient, procedure, or geographic location requires consulting a complex and constantly evolving set of policy documents. These documents are maintained by the Centers for Medicare & Medicaid Services (CMS) and exist in two primary forms: tNational Coverage Determinations (NCDs), which establish federal-level coverage applicable uniformly across all states, and *Local Coverage Determinations* (LCDs), which define regional rules set by Medicare Administrative Contractors (MACs) for specific jurisdictions.

In practice, a provider trying to answer a question such as *"Does Medicare cover physical therapy after knee replacement in Texas?"* must manually search through hundreds of documents, cross-reference national and state-level policies, and interpret dense regulatory prose. This process is time-consuming, error-prone, and inaccessible to non-specialists. The consequences of misinterpretation are significant: incorrect billing leads to claim denials, and uninformed patients may forgo covered treatments.

Large Language Models (LLMs) offer a promising path toward automating policy question answering. However, general-purpose LLMs suffer from a critical limitation in this context: *hallucination* — the tendency to generate plausible-sounding but factually incorrect information. In the healthcare domain, a hallucinated coverage determination can have serious real-world consequences.

Retrieval-Augmented Generation (RAG) addresses this by grounding the model's responses in retrieved documents from a curated knowledge base. Rather than relying on parametric knowledge from pre-training, a RAG system retrieves relevant policy chunks at query time and passes them to the LLM as context, instructing it to generate answers only from the provided evidence. This dramatically reduces hallucination while enabling precise, citable responses.

This project implements a complete RAG pipeline for Medicare policy Q&A using real CMS data, evaluates it against an ungrounded LLM baseline, and delivers an interactive demo via a Streamlit interface.

2. Literature Review

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation was formally introduced by Lewis et al. (2020) as a general framework for knowledge-intensive NLP tasks [1]. The key insight is that combining a parametric memory (the LLM) with a non-parametric memory (a retrieval index over documents) produces systems that are more factually accurate, more interpretable, and more easily updated than fine-tuned models alone.

Subsequent work has refined each pipeline component. Karpukhin et al. (2020) demonstrated that dense bi-encoder retrieval substantially outperforms BM25 sparse retrieval for open-domain question answering [2]. Nogueira and Cho (2019) introduced neural reranking via BERT-based cross-

encoders, showing that a two-stage retrieve-then-rerank pipeline significantly improves precision over single-stage retrieval [3]. This design — fast approximate retrieval followed by accurate reranking — has become the standard for production RAG systems and directly informs our architecture.

2.2 RAG in Healthcare

The healthcare domain presents unique NLP challenges: terminology is highly specialized, documents are long and structurally complex, and factual accuracy carries direct clinical and legal implications. Xiong et al. (2024) demonstrated that RAG systems significantly outperform standard LLMs on medical QA benchmarks, particularly on questions requiring specific factual recall [4]. Crucially, they found that retrieval quality — specifically whether the correct source document appeared in the top-k candidates — was the dominant factor in system performance, motivating our emphasis on reranking.

Singhal et al. (2023) showed that domain-adapted LLMs with grounded generation achieve expert-level clinical QA performance, but their approach required expensive fine-tuning on proprietary data [5]. Our work pursues a complementary strategy: rather than adapting the model, we focus on retrieval quality and grounding via prompting, making the system more accessible and easily updated as policies change.

Narayanan et al. (2023) applied RAG to health insurance document Q&A and found that chunking strategy had a significant effect on retrieval quality, with section-aware chunking outperforming naive fixed-length splitting [6]. This finding directly informed our chunking design decisions (see Section 5).

2.3 Embedding Models and Rerankers

Xiao et al. (2023) introduced the BGE (BAAI General Embedding) model family, achieving state-of-the-art performance on the MTEB benchmark across retrieval, reranking, and semantic similarity tasks [7]. BGE models are open-source, freely available on HuggingFace, and optimized for asymmetric retrieval where queries and documents differ in length — exactly our setting. We adopt `BAAI/bge-base-en-v1.5` for dense retrieval and `BAAI/bge-reranker-v2-m3` for cross-encoder reranking.

2.4 Hallucination and Faithfulness

Maynez et al. (2020) provide a taxonomy of hallucination distinguishing *intrinsic* errors (contradicting the source) from *extrinsic* errors (introducing absent information) [8]. Min et al. (2023) showed that faithfulness — the degree to which a generated answer is supported by retrieved evidence — can be reliably measured using an LLM-as-judge methodology [9]. We adopt both human annotation and LLM-judge evaluation in our experiments.

3. Project Scope and Objectives

3.1 Problem Statement

We address the following problem: *given a natural language question about Medicare coverage, generate a factually accurate, grounded, and citable answer derived from official CMS policy documents.*

Representative questions the system is designed to answer include:

- *"Does Medicare cover acupuncture for chronic lower back pain?"*
- *"Is home oxygen therapy covered for patients in Texas?"*
- *"What are the criteria for cardiac rehabilitation coverage in Florida?"*
- *"Does Medicare cover continuous glucose monitoring devices?"*
- *"What wound care treatments are covered under Novitas?"*

3.2 Objectives

The specific objectives of this project are:

1. Build a curated knowledge base from real CMS NCD and LCD documents.
2. Implement a two-stage retrieval pipeline: dense embedding retrieval followed by cross-encoder reranking.
3. Integrate retrieved evidence with an open-source LLM to produce grounded, cited answers.
4. Evaluate the system on retrieval quality, answer correctness, hallucination rate, and citation accuracy.
5. Compare three configurations: Baseline LLM, RAG, and RAG with query rewriting.
6. Deliver a functional interactive demo via Streamlit.

3.3 Scope Boundaries

This project focuses exclusively on Medicare coverage policy. We do not address Medicaid, private insurance, or clinical treatment guidelines. The knowledge base is restricted to NCDs and LCDs sourced from CMS as of early 2026. The system is a decision-support tool and does not constitute medical or legal advice.

4. Data Collection and Preprocessing

4.1 Overview of Data Sources

All data was sourced from the **Centers for Medicare & Medicaid Services (CMS)**, the federal agency responsible for administering Medicare. We collected four distinct document corpora, each serving a different role in answering Medicare coverage questions. Two were obtained as structured bulk exports from the CMS Medicare Coverage Database:

https://www.cms.gov/medicare-coverage-database/downloads/downloads.aspx

Two additional sources — the Medicare Benefit Policy Manual and the Medicare Claims Process-

ing Manual — were obtained as PDF chapter downloads from the CMS Internet-Only Manuals repository:

https://www.cms.gov/Regulations-and-Guidance/Guidance/Manuals/Internet-Only-Manuals-IOMs

Together, these four sources provide comprehensive coverage of Medicare policy at both the determination level (what is covered) and the operational level (how coverage is defined and administered). Each source is described in detail below.

4.2 National Coverage Determinations (NCDs)

4.2.1 What NCDs Are

A National Coverage Determination is a federal-level ruling by CMS on whether a particular item, service, treatment, or technology is covered under Medicare — and if so, under what clinical conditions. NCDs are binding across all 50 states and all Medicare Administrative Contractors. They are issued after a formal evidence review process and represent the most authoritative statement of Medicare coverage for a given topic.

Each NCD is identified by a section number in the *Medicare National Coverage Determinations Manual* (CMS Publication 100-03), for example:

- **NCD 30.1** — Acupuncture for Chronic Low Back Pain (added 2020)
- **NCD 100.3** — 24-Hour Ambulatory Esophageal pH Monitoring
- **NCD 160.18** — Continuous Glucose Monitors
- **NCD 20.4** — Implantable Cardiac Pacemakers
- **NCD 80.12** — Positron Emission Tomography (PET) Scans

A typical NCD specifies the clinical indications under which coverage applies, any limitations or exclusions, and references the clinical evidence supporting the determination. For example, NCD 30.1 (Acupuncture) covers up to 12 visits within 90 days for chronic low back pain lasting 12 or more weeks, with an additional 8 sessions if the patient demonstrates measurable improvement.

4.2.2 How We Downloaded NCD Data

The full NCD dataset was obtained as a structured bulk export from the CMS Downloads page. The export consisted of four CSV files:

- `ncd_trkg.csv` — the primary table containing all NCD records with full HTML-formatted policy text
- `ncd_pblctn_ref.csv` — a reference table mapping publication codes to manual section names
- `ncd_bnft_ctgry_ref.csv` — benefit category reference (e.g., "Home Health", "Durable Medical Equipment")
- `ncd_trkg_bnft_xref.csv` — a cross-reference linking each NCD to its benefit categories

We parsed `ncd_trkg.csv` as the primary source, filtering to active records (`status = 'A'`) and

removing duplicates by retaining the most recent version of each NCD. After filtering, we extracted 343 unique active NCDs. The policy text was HTML-formatted; we used Python's `BeautifulSoup` library to strip all HTML tags and extract clean plain text. Each NCD was saved as a structured plain-text file:

```
TITLE: Acupuncture for Chronic Low Back Pain
SECTION: 30.1
EFFECTIVE DATE: 2020-01-21
COVERAGE POLICY:
Medicare covers acupuncture for chronic low back pain (cLBP)
lasting 12 weeks or longer; not associated with surgery...
```

4.3 Local Coverage Determinations (LCDs)

4.3.1 What LCDs Are

While NCDs establish national-level policy, *Local Coverage Determinations* are issued by Medicare Administrative Contractors (MACs) — private companies contracted by CMS to administer Medicare claims in specific geographic regions. Each MAC issues LCDs applicable to the states it serves, meaning coverage for a given procedure may differ by location even when no national NCD exists.

The United States is divided into several MAC jurisdictions. Of particular relevance to our corpus is **Novitas Solutions**, which serves Jurisdiction H (Texas, Arkansas, Colorado, Louisiana, Mississippi, New Mexico, and Oklahoma) and Jurisdiction L (Delaware, Maryland, New Jersey, Pennsylvania, and Washington DC). A user asking *"Does Medicare cover wound care in Texas?"* may receive a more precise answer from a Novitas LCD than from any federal NCD. Our system preserves and leverages this state metadata throughout the pipeline.

4.3.2 Examples of LCDs in Our Corpus

Table 1: Sample LCD Documents in the Corpus

LCD ID	Title	Contractor	States (sample)
L35125	Wound Care	Novitas	TX, AR, CO, LA
L33797	Oxygen and Oxygen Equipment	CGS	KY, OH
L37792	4Kscore Test Algorithm	National	All states
L35062	Physical Therapy	Novitas	TX, AR, CO
L37387	Chiropractic Services	WPS	WI, MI, MN
L35050	Outpatient Sleep Studies	Palmetto	SC, NC, VA

4.3.3 How We Downloaded LCD Data

The LCD bulk dataset was downloaded from the CMS Downloads page as a single Excel file (`lcd_half.xlsx`). The file presented a significant formatting challenge: the HTML-encoded policy text had overflowed across hundreds of Excel columns during export, producing a table with over 1,300 columns that standard tools could not parse correctly.

We resolved this programmatically using Python's `openpyxl` library, which handles malformed column structures gracefully. After filtering to valid LCD records (numeric `lcd_id`, status `'A'`, and

title length under 300 characters) and deduplicating by LCD ID to retain the most recent version of each policy, we obtained **819 active LCD records**.

A critical preprocessing step was **state attribution**: mapping each LCD to the states it governs based on CMS's official MAC jurisdiction assignments. Each file header was enriched with contractor and state metadata:

```
TITLE: Wound Care
LCD_ID: 35125
CONTRACTOR: Novitas
STATES: TX, AR, CO, LA, MS, NM, OK, DE, DC, MD, NJ, PA
EFFECTIVE_DATE: 2018-04-19

COVERAGE POLICY:
Medicare covers wound care services when the patient presents
with a chronic wound that has not responded to standard
treatment after 30 days...
```

This state metadata is preserved in every downstream chunk, enabling state-aware retrieval — so a Texas-specific query automatically prioritizes chunks tagged with `TX`.

4.4 Medicare Benefit Policy Manual (Publication 100-02)

4.4.1 What the Benefit Policy Manual Is

The *Medicare Benefit Policy Manual* is CMS Publication 100-02 and serves as the structural foundation for understanding what Medicare covers and under what conditions. While NCDs and LCDs answer "is procedure X covered?", the Benefit Policy Manual answers the more fundamental question: "what does Medicare coverage actually mean?" It defines core concepts such as medical necessity, skilled nursing care, homebound status, and the conditions under which different benefit categories apply.

The manual is organized into 17 chapters, each covering a distinct benefit category:

- **Chapter 1** — Inpatient Hospital Services
- **Chapter 6** — Hospital Services Covered Under Part B
- **Chapter 7** — Home Health Services
- **Chapter 8** — Skilled Nursing Facility (SNF) Coverage
- **Chapter 9** — Hospice Services
- **Chapter 10** — Ambulance Services
- **Chapter 11** — End Stage Renal Disease (ESRD)
- **Chapter 15** — Covered Medical and Other Health Services
- **Chapter 16** — General Exclusions from Coverage
- *... and 8 additional chapters*

Chapter 16 (General Exclusions from Coverage) is particularly valuable for our system, as it enumerates categories of services that Medicare explicitly does not cover — enabling the system to correctly answer negative questions such as *"Does Medicare cover routine dental care?"* (it does not).

4.4.2 How We Downloaded the Benefit Policy Manual

All 17 chapters were downloaded as individual PDF files from the CMS Internet-Only Manuals repository. Text was extracted from each PDF using the `PyMuPDF` (`fitz`) library, which preserves paragraph structure and handles the multi-column layouts common in CMS documents. Each chapter was tagged with its manual name, chapter title, and a scope of `STATES: ALL` since benefit policy applies nationally.

4.5 Medicare Claims Processing Manual (Publication 100-04)

4.5.1 What the Claims Processing Manual Is

The *Medicare Claims Processing Manual* is CMS Publication 100-04 and governs how Medicare claims are submitted, processed, and adjudicated. While primarily administrative in nature, it contains substantial coverage-relevant content: it specifies billing codes, documentation requirements, and the exact conditions under which specific services are reimbursable. For a RAG system answering questions from healthcare providers, this manual is important because providers frequently need to know not just whether something is covered, but what documentation is required to support a claim.

The manual spans 39 chapters covering the full range of Medicare claim types:

- **Chapter 1** — General Billing Requirements
- **Chapter 10** — Home Health Agency Billing
- **Chapter 11** — Processing Hospice Claims
- **Chapter 12** — Physicians and Nonphysician Practitioners
- **Chapter 16** — Laboratory Services
- **Chapter 18** — Preventive and Screening Services
- **Chapter 20** — Durable Medical Equipment (DMEPOS)
- *... and 32 additional chapters*

4.5.2 How We Downloaded the Claims Processing Manual

All 39 chapters were downloaded as individual PDF files from the same CMS Internet-Only Manuals repository and processed identically to the Benefit Policy Manual — PDF text extraction via `PyMuPDF`, chunked at 400 words with 50-word overlap, and tagged with `MANUAL: Medicare Claims Processing Manual`, chapter title, and `STATES: ALL`.

4.6 Final Corpus Summary

Table 2: Complete Data Collection Summary

Document Type	Format	Source	Count
National Coverage Determinations	CSV bulk export	CMS Coverage Database	343 docs
Local Coverage Determinations	Excel bulk export	CMS Coverage Database	819 docs
Medicare Benefit Policy Manual	PDF (per chapter)	CMS IOMs	17 chapters
Medicare Claims Processing Manual	PDF (per chapter)	CMS IOMs	39 chapters
Total			**1,218 documents**

The four sources are complementary by design. NCDs and LCDs answer specific coverage questions at the policy level. The Benefit Policy Manual provides the structural definitions and benefit rules that underpin those policies. The Claims Processing Manual adds the provider-facing operational layer: documentation requirements, billing codes, and submission criteria. Together they form a knowledge base that can answer the full range of Medicare coverage questions a provider or patient might ask.

5. Step 1: Document Chunking and Corpus Construction

5.1 Why Chunking Is Necessary

A fundamental challenge in building a retrieval system over long documents is that embedding models encode a fixed-length vector for each input. A full NCD or LCD document may contain 1,000–3,000 words: if the entire document is encoded as a single vector, semantically distinct sections average together and retrieval precision degrades. Similarly, passing complete documents to an LLM at generation time is inefficient and may exceed context window limits.

The standard solution is to split documents into smaller, overlapping *chunks* before indexing. Each chunk is independently embedded and stored. At query time, only the most relevant chunks are retrieved and passed to the LLM — not entire documents.

5.2 Chunking Strategy and Parameters

We adopted a **fixed-size sliding window** approach with the following parameters:

- **Chunk size**: 400 words
- **Overlap**: 50 words between consecutive chunks

The 400-word size balances retrieval granularity and semantic coherence. Chunks shorter than 200 words often lack sufficient context for the embedding model to capture policy meaning. Chunks longer than 600 words begin to mix multiple distinct coverage criteria, reducing retrieval precision. The 50-word overlap ensures that sentences near chunk boundaries appear in at least two chunks, preventing information loss at split points.

5.3 Metadata Tagging Design

A critical design decision was to embed document metadata *directly into every chunk's text*, rather than storing it only as a separate database field. This ensures the LLM always has access to citation information regardless of how the chunk is retrieved. Every chunk is prefixed with a structured metadata header:

For NCD chunks:

```
[TYPE: NCD | NCD_ID: 30.1 | TITLE: Acupuncture for Chronic Low Back Pain]
```

For LCD chunks:

```
[TYPE: LCD | LCD_ID: 35125 | TITLE: Wound Care |
```

```
STATES: TX, AR, CO, LA | CONTRACTOR: Novitas]
```

This allows the LLM to generate citations such as *"According to LCD L35125 (Wound Care, Novitas Solutions)..."* directly from the chunk content, without any additional lookup.

5.4 Implementation

The chunking pipeline was implemented in Python and executed in Google Colab Pro. The core chunking function is:

```python
def chunk_text(text, chunk_size=400, overlap=50):
    words = text.split()
    chunks = []
    i = 0
    while i < len(words):
        chunk = ' '.join(words[i:i + chunk_size])
        chunks.append(chunk)
        i += chunk_size - overlap
        if i + overlap >= len(words):
            break
    return chunks
```

Listing 1: Core sliding-window chunking function

Metadata is parsed from the structured file headers and prepended to each chunk. The LCD parser, which also extracts state information, is shown below:

```python
def parse_lcd_file(filepath, filename):
    with open(filepath, encoding='utf-8') as f:
        content = f.read()

    lines = content.split('\n')
    title, lcd_id, contractor = '', 'UNKNOWN', 'UNKNOWN'
    states = ['ALL']

    for line in lines[:15]:
        if line.startswith('TITLE:'):
            title = line.replace('TITLE:', '').strip()
        elif line.startswith('LCD_ID:'):
            lcd_id = line.replace('LCD_ID:', '').strip()
        elif line.startswith('CONTRACTOR:'):
            contractor = line.replace('CONTRACTOR:', '').strip()
        elif line.startswith('STATES:'):
            raw = line.replace('STATES:', '').strip()
            states = [s.strip() for s in raw.split(',')]

    text_chunks = chunk_text(content)
```

```python
    results = []

    for i, chunk in enumerate(text_chunks):
        header = (
            f"[TYPE: LCD | LCD_ID: {lcd_id} | TITLE: {title}"
            f" | STATES: {', '.join(states)}"
            f" | CONTRACTOR: {contractor}]"
        )
        tagged_chunk = f"{header}\n\n{chunk}"
        results.append({
            'text':       tagged_chunk,
            'raw_text':   chunk,
            'source_id':  lcd_id,
            'title':      title,
            'type':       'LCD',
            'states':     states,
            'contractor': contractor,
            'filename':   filename,
            'chunk_idx':  i
        })
    return results
```

Listing 2: LCD file parser with state metadata extraction

All 1,218 processed documents were merged into a single `all_chunks.json` file, where each entry follows this schema:

```json
{
    "text":       "[TYPE: LCD | LCD_ID: 35125 | ...]\n\nMedicare covers...",
    "raw_text":   "Medicare covers wound care when the patient...",
    "source_id":  "35125",
    "title":      "Wound Care",
    "type":       "LCD",
    "states":     ["TX", "AR", "CO", "LA"],
    "contractor": "Novitas",
    "filename":   "Wound_Care_35125.txt",
    "chunk_idx":  0
}
```

Listing 3: JSON schema for each chunk in all_chunks.json

5.5 Corpus Statistics

The substantially higher chunk counts for the two CMS manuals reflect their greater length and scope: each PDF chapter spans dozens of pages of detailed policy, billing codes, and administrative guidance, compared to the more concise NCD and LCD documents. The complete corpus of 8,563

Table 3: Corpus Statistics After Chunking

Document Type	Files	Chunks	Avg. Chunks / Doc
National Coverage Determinations	343	575	1.68
Local Coverage Determinations	819	2,802	3.42
Medicare Benefit Policy Manual	17	1,039	61.1
Medicare Claims Processing Manual	39	4,147	106.3
Total	**1,218**	**8,563**	**7.03**

chunks in `all_chunks.json` serves as input to all downstream pipeline components.

6. Step 2: Embeddings and Vector Index

6.1 Embedding Model Selection

To convert text chunks into dense vector representations suitable for semantic search, we selected `BAAI/bge-base-en-v1.5` from the BGE (BAAI General Embedding) model family. This model was chosen for several reasons:

- **Retrieval benchmark performance**: BGE models achieve state-of-the-art scores on the Massive Text Embedding Benchmark (MTEB) across retrieval, reranking, and semantic similarity tasks, outperforming other base-sized models at the time of selection.
- **Asymmetric retrieval optimization**: BGE is specifically designed for settings where queries are short (user questions) and documents are longer (policy chunks). This asymmetry is a poor fit for models trained primarily on symmetric sentence similarity.
- **768-dimensional output**: The base variant produces 768-dimensional embeddings, providing a strong balance between representation quality and computational efficiency. Larger variants (e.g., `bge-large`) offer marginal gains at significantly higher cost.
- **Open-source availability**: The model is freely available on HuggingFace under the MIT license, with no API costs which is important for a reproducible academic project.

6.2 Encoding the Corpus

All 8,563 chunks from `all_chunks.json` were encoded into dense vectors using the `sentence-transformers` library. We embedded the **metadata-tagged `text` field** (not the `raw_text` field) from each chunk. This is an intentional design decision: the metadata header (e.g., `[TYPE: LCD | LCD_ID: 35125 | TITLE: Wound Care | STATES: TX, AR, CO, LA | CONTRACTOR: Novitas]`) is semantically informative. By embedding it alongside the policy content, a query about "wound care Texas" can match on both the policy text and the metadata tags, improving retrieval quality.

A critical encoding parameter is **L2 normalization**: each embedding vector is normalized to unit length during encoding (`normalize_embeddings=True`). This ensures that the inner product between any two vectors equals their cosine similarity, which simplifies the FAISS index configuration (see below). After encoding, we verified that all 8,563 vectors had an L2 norm of exactly 1.0.

The encoding was performed on a GPU-enabled Google Colab instance with a batch size of 64. The core encoding logic is:

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer('BAAI/bge-base-en-v1.5', device='cuda')

texts = [chunk['text'] for chunk in all_chunks]

embeddings = model.encode(
    texts,
    batch_size=64,
    show_progress_bar=True,
    normalize_embeddings=True,
    convert_to_numpy=True
)
# embeddings.shape: (8563, 768), dtype: float32
```

Listing 4: Encoding all chunks into normalized dense vectors

6.3 FAISS Index Construction

We use **FAISS** (Facebook AI Similarity Search) to index the embedding vectors for fast nearest-neighbor retrieval. FAISS is a library developed by Meta AI for efficient similarity search over dense vectors, supporting both exact and approximate search algorithms.

We selected `IndexFlatIP` (flat inner product index) for the following reasons:

1. **Exact search**: With approximately 8,500 vectors, brute-force search completes in 1–5ms per query which is fast enough for interactive use. Approximate methods (IVF, HNSW) add complexity without meaningful speedup at this scale.
2. **Cosine similarity via inner product**: Since all embeddings are L2-normalized, the inner product between any two vectors equals their cosine similarity. This avoids a separate normalization step at query time.
3. **No training required**: Unlike approximate indexes such as `IndexIVFFlat`, `IndexFlatIP` requires no training or clustering step where vectors are simply added to the index.
4. **Deterministic results**: Every query returns the mathematically exact top-k results, ensuring full reproducibility.

The index construction code is straightforward:

```
import faiss

dimension = embeddings.shape[1]  # 768
index = faiss.IndexFlatIP(dimension)
index.add(embeddings.astype('float32'))
```

```

# Save to Google Drive
faiss.write_index(index, 'faiss_index/medicare.index')
```

Listing 5: Building and saving the FAISS index

6.4 Index Statistics

Table 4: Embedding and Index Statistics

Metric	Value
Total chunks indexed	8,563
Embedding model	`BAAI/bge-base-en-v1.5`
Embedding dimension	768
Normalization	L2-normalized
Similarity metric	Cosine (via inner product)
FAISS index type	`IndexFlatIP` (exact)
Index file size	25.1 MB
Embeddings file size	25.1 MB
Metadata file size	4.0 MB

Four artifacts were saved to Google Drive for use by downstream pipeline components: the FAISS index file (`medicare.index`), a document-ID mapping (`docids.txt`), the raw NumPy embedding array (`embeddings.npy`), and a chunk metadata JSON file (`chunk_metadata.json`) containing the type, title, states, contractor, and filename for each indexed chunk.

6.5 Retrieval Sanity Test

Before handing off to the reranking pipeline, we verified that the FAISS index retrieves semantically relevant chunks by running three sample queries spanning different document types. For each query, the top 3 results (before any reranking) are shown below.

Query 1 (NCD — national coverage): *"Does Medicare cover acupuncture for chronic lower back pain?"*

Rank	Type	ID	Title	Score
1	NCD	30.3.3	Acupuncture for Chronic Lower Back Pain (cLBP)	0.8095
2	NCD	30.3	Acupuncture	0.7742
3	NCD	30.3.1	Acupuncture for Fibromyalgia	0.7358

The top result is the exact NCD governing acupuncture for chronic lower back pain, confirming that the embedding model captures the semantic match between the query and the policy document.

Query 2 (LCD — state-specific): *"Is home oxygen therapy covered for patients in Texas?"*

Rank	Type	ID	Title	Score
1	NCD	240.2	Home Use of Oxygen	0.8005
2	NCD	240.2	Home Use of Oxygen (chunk 2)	0.7805
3	NCD	240.2.1	Home Use of Oxygen in Clinical Trials	0.7583

The retrieval correctly identifies the NCD governing home oxygen use. Notably, the LCD for "Oxygen and Oxygen Equipment" (LCD 33797, which covers Texas) appears at rank 4 with a score of 0.7432. This illustrates a limitation of dense retrieval alone: the NCD is semantically closer to the query, but for a Texas-specific question the LCD may be more operationally relevant. State filtering and reranking (Step 3) address this.

Query 3 (Benefit Policy): *"What are the requirements for skilled nursing facility coverage?"*

Rank	Type	ID	Title	Score
1	Benefit Policy	Ch. 7	Home Health Services	0.7718
2	Benefit Policy	Ch. 8	Coverage of Extended Care (SNF)	0.7687
3	Benefit Policy	Ch. 8	Coverage of Extended Care (SNF)	0.7645

Chapter 8 (SNF Coverage) appears at ranks 2 and 3, confirming that the embeddings successfully capture the relationship between "skilled nursing facility" in the query and "extended care (SNF)" in the manual chapter titles. Chapter 7 (Home Health) at rank 1 reflects the semantic overlap between home health and SNF eligibility criteria — both involve skilled care requirements.

These results demonstrate that the FAISS index provides strong semantic retrieval across all four document types in the corpus. The next pipeline stage (Step 3) adds state-aware filtering and cross-encoder reranking to further improve precision.

7. Step 3: Retrieval Pipeline and Reranking

In order to retrieve the most relevant Medicare policy information for a user query, MediQuery implements a two-stage retrieval pipeline commonly used in modern Retrieval-Augmented Generation (RAG) systems. This approach combines fast semantic retrieval using dense embeddings with a more precise neural reranking step.

7.1 Two-Stage Retrieve-Then-Rerank Architecture

The retrieve-then-rerank architecture is designed to balance efficiency and accuracy. Searching across thousands of document chunks using a highly accurate neural model would be computationally expensive. Instead, the system first performs fast approximate retrieval to identify a small candidate set of potentially relevant documents. A more computationally intensive model is then applied to refine the ranking of those candidates.

The architecture used in MediQuery follows the pipeline shown below:

$$\text{Query} \rightarrow \text{Embedding Model} \rightarrow \text{FAISS Retrieval} \rightarrow \text{Candidate Chunks} \rightarrow \text{Cross-Encoder Reranker} \rightarrow \text{Top Evidence Chunks} \rightarrow \text{LLM}$$

This architecture ensures that the system remains responsive while still providing highly relevant evidence for answer generation.

7.2 Stage 1: Dense Retrieval

The first stage performs semantic retrieval using dense vector embeddings. When a user submits a question, the system encodes the query using the same embedding model used to encode the document corpus, `BAAI/bge-base-en-v1.5`. This ensures that both queries and documents are represented in the same semantic vector space.

Once the query embedding is generated, the system performs nearest-neighbor search in the FAISS index containing all document embeddings. The index used in this project is `IndexFlatIP`, which performs similarity search using inner product operations. Because all embeddings are normalized, inner product corresponds to cosine similarity.

The FAISS search returns the top 20 document chunks that are most similar to the query embedding. These chunks form the initial candidate set of potentially relevant evidence.

7.3 State-Aware Filtering

Medicare policies often vary by geographic region because Local Coverage Determinations (LCDs) are issued by Medicare Administrative Contractors that cover specific states. During preprocessing, each document chunk was tagged with metadata including the states covered by that policy.

After retrieving the candidate chunks from FAISS, the system applies a state-aware filtering step. If a user query mentions a specific state (for example Texas), the system prioritizes chunks that contain the corresponding state code in their metadata. For example, a query related to Texas will prioritize chunks tagged with `TX`. This filtering step ensures that the system retrieves policies that are relevant to the user's geographic context.

7.4 Stage 2: Cross-Encoder Reranking

While dense retrieval is efficient, it can sometimes return documents that are semantically related but not directly relevant to the user's question. To address this limitation, MediQuery applies a neural reranking model to the candidate set.

The reranking model used in this project is `BAAI/bge-reranker-v2-m3`. This model evaluates the relevance of each query-document pair by jointly processing the query and the document text.

For each retrieved chunk, the system constructs a pair consisting of the query and the chunk text. The reranker assigns a relevance score to each pair. The candidate chunks are then sorted according to their reranker scores, and the top five chunks are selected as the final evidence set.

7.5 Bi-Encoder vs Cross-Encoder Models

Dense retrieval models such as BGE embeddings use a bi-encoder architecture. In a bi-encoder, the query and document are encoded independently into vectors, and similarity is computed using vector operations. This approach is very fast and allows pre-computation of document embeddings, making it suitable for large-scale retrieval.

However, because the query and document are encoded separately, the model cannot fully capture detailed interactions between the two pieces of text. As a result, bi-encoder retrieval can sometimes produce approximate rankings.

Cross-encoders address this limitation by processing the query and document together as a single input sequence. This allows the model to capture richer contextual relationships between the query and the document text. The trade-off is that cross-encoders are significantly slower because they must evaluate each query-document pair individually.

By combining both approaches, the MediQuery system benefits from the speed of bi-encoder retrieval and the accuracy of cross-encoder reranking.

7.6 Retrieve and Rerank Implementation

The retrieval pipeline is implemented using a Python function that retrieves candidate chunks from the FAISS index and then reranks them using the cross-encoder model.

```python
def rerank_results(query, results, top_n=5):

    pairs = [(query, r["text"]) for r in results]

    scores = reranker.predict(pairs)

    for i in range(len(results)):
        results[i]["rerank_score"] = float(scores[i])

    results.sort(
        key=lambda x: x["rerank_score"],
        reverse=True
```

```
    )

    seen = set()
    final = []

    for r in results:
        if r["source_id"] not in seen:
            final.append(r)
            seen.add(r["source_id"])

        if len(final) == top_n:
            break

    return final
```

This function computes relevance scores for each retrieved chunk and returns the top ranked chunks after removing duplicate source documents.

7.7 Before and After Reranking Comparison

Query	Top Result Before Reranking	Top Result After Reranking
Does Medicare cover acupuncture for chronic lower back pain?	General pain management guideline related to acupuncture	Medicare policy section covering acupuncture for chronic lower back pain
Does Medicare cover wound care in Texas?	National wound care overview document	Texas Local Coverage Determination (LCD) policy for wound care services

Table 5: Example comparison of retrieval results before and after reranking

To evaluate the impact of reranking, we compared the top retrieved documents before and after applying the cross-encoder model.

The results show that reranking improves retrieval precision by promoting policy chunks that more directly answer the user's question.

7.8 Fine-Tuning the Bi-Encoder

While the pre-trained BGE embedding model provides strong general-purpose retrieval, its representations are not optimized for the specialized vocabulary and structure of Medicare policy documents. To close this domain gap, we fine-tuned both the bi-encoder and the cross-encoder reranker on Medicare-specific training data derived from our corpus.

7.8.1 Training Data Construction

We constructed a training set from the 56,288 synthetic query-gold chunk pairs described in Section 5. Each pair links a natural-language question to its source chunk in `all_chunks.json`. The positive passage for each training example is the **full indexed chunk text** (including the metadata header),

not the short excerpt from the evaluation CSV. This ensures that the fine-tuned model learns representations consistent with the actual documents it will search at inference time.

7.8.2 Hard Negative Mining

Effective contrastive learning requires not only positive pairs but also challenging negative examples that the model currently confuses with true matches. For each of the 56,288 queries, we encoded the query with the pre-trained `bge-base-en-v1.5` model and retrieved the top 300 candidate chunks from the pre-trained FAISS index. The hard negative was selected as the **highest-ranked chunk whose source document ID differs** from the gold chunk's source ID. This guarantees that each negative is semantically close to the query but originates from a different policy document. All 56,288 pairs received valid hard negatives.

7.8.3 Train / Validation / Test Split

The 56,288 triplets were split in two stages, both stratified by query type to maintain label balance:

1. 10% held out as a **test set** (5,629 examples) for final evaluation.
2. The remaining 90% split 95/5 into **train** (48,126) and **validation** (2,533).

7.8.4 Loss Function and Hyperparameters

We fine-tuned `BAAI/bge-base-en-v1.5` using **MultipleNegativesRankingLoss** (MNRL) with a scale factor of 50.0 (equivalent to a contrastive temperature of 0.02), matching the temperature used in BGE's original pre-training. Each training batch contained 32 unique anchors; the `NO_DUPLICATES` batch sampler ensured maximum in-batch negative diversity.

Table 6: Bi-Encoder Fine-Tuning Hyperparameters

Parameter	**Value**
Base model	`BAAI/bge-base-en-v1.5`
Epochs	2
Batch size	32
Learning rate	1×10^{-5}
Warmup steps	300 (10% of 3,006 total)
Temperature	0.02 (scale = 50)
Precision	fp16
Batch sampler	`NO_DUPLICATES`
Best-model metric	`cosine_ndcg@10`

Validation was performed every 500 steps using an `InformationRetrievalEvaluator` that searched 2,533 validation queries against the full 8,563-chunk corpus. The best checkpoint was selected by cosine NDCG@10.

The core training setup is shown below:

```
from sentence_transformers import SentenceTransformer
from sentence_transformers.losses import MultipleNegativesRankingLoss
from sentence_transformers.training_args import BatchSamplers

```

```
biencoder_model = SentenceTransformer('BAAI/bge-base-en-v1.5')
train_loss = MultipleNegativesRankingLoss(biencoder_model, scale=50.0)

args = SentenceTransformerTrainingArguments(
    output_dir=FT_BIENCODER_DIR,
    num_train_epochs=2,
    per_device_train_batch_size=32,
    learning_rate=1e-5,
    warmup_steps=300,
    fp16=True,
    batch_sampler=BatchSamplers.NO_DUPLICATES,
    load_best_model_at_end=True,
    metric_for_best_model="cosine_ndcg@10",
)
trainer = SentenceTransformerTrainer(
    model=biencoder_model, args=args,
    train_dataset=biencoder_train_dataset,
    loss=train_loss,
    evaluator=biencoder_evaluator,
)
trainer.train()
```

Listing 6: Bi-encoder fine-tuning with MNRL loss

7.8.5 Re-Encoding and Index Rebuild

After fine-tuning, all 8,563 chunks were re-encoded with the fine-tuned bi-encoder (`batch_size=64`, `normalize_embeddings=True`). A new FAISS `IndexFlatIP` index was built from the fine-tuned embeddings and saved as `medicare_finetuned.index` (25.1 MB).

7.9 Fine-Tuning the Cross-Encoder Reranker

7.9.1 Hard Negative Re-Mining

Because the fine-tuned bi-encoder produces a different retrieval ranking than the pre-trained model, hard negatives for reranker training were **re-mined from the fine-tuned FAISS index**. For each of the 48,126 training queries, we retrieved the top 100 candidates from the fine-tuned index and selected up to 7 hard negatives per query, each from a distinct source document. This yields a training group size of 8 per query (1 positive + 7 negatives), following the FlagEmbedding training protocol. After filtering, the reranker training set contained 307,877 pairwise examples.

7.9.2 Loss Function and Hyperparameters

We fine-tuned `BAAI/bge-reranker-v2-m3` using **BinaryCrossEntropyLoss** for pairwise relevance classification. Each example consisted of a query-chunk pair labeled 1.0 (positive) or 0.0 (negative).

Validation used a `CrossEncoderRerankingEvaluator` with 2,042 validation samples, each containing 1 positive and its corresponding hard negatives. The best checkpoint was selected by

Table 7: Reranker Fine-Tuning Hyperparameters

Parameter	**Value**
Base model	`BAAI/bge-reranker-v2-m3`
Epochs	2
Batch size	32
Learning rate	1×10^{-5}
Warmup steps	10% of total steps
Weight decay	0.01
Precision	fp16
Gradient checkpointing	Enabled
Max sequence length	2,048 tokens
Best-model metric	`ndcg@5`

NDCG@5.

7.10 Retrieval Improvement After Fine-Tuning

To quantify the impact of domain-specific fine-tuning, we evaluated both the pre-trained and fine-tuned pipelines on the held-out test set of 5,629 queries. FAISS retrieval returned the top 20 candidates; the reranker selected the top 5 from the top 20 FAISS results.

Table 8: Retrieval Comparison: Pre-trained vs. Fine-tuned Pipeline

Metric	**Pre-trained**	**Fine-tuned**	**Delta**
Recall@5 (FAISS)	0.6460	0.7850	+0.1390
Recall@10 (FAISS)	0.7260	0.8420	+0.1160
Recall@20 (FAISS)	0.8040	0.9010	+0.0970
MRR (FAISS)	0.4992	0.6120	+0.1128
NDCG@5 (FAISS)	0.5224	0.6450	+0.1226
NDCG@10 (FAISS)	0.5488	0.6730	+0.1242
NDCG@20 (FAISS)	0.5686	0.6910	+0.1224
Recall@5 (reranked)	0.6720	0.8100	+0.1380
MRR (reranked)	0.4751	0.6300	+0.1549
NDCG@5 (reranked)	0.5244	0.6650	+0.1406

Fine-tuning produced consistent improvements across every metric. The largest gain was in reranked MRR (+0.1549), indicating that fine-tuning not only increases whether the correct chunk is retrieved but also how high it ranks among the final candidates shown to the generator. Reranked Recall@5 reached 0.8100, meaning the correct evidence chunk appears in the top-5 context for over 81% of test queries.

7.10.1 Impact of Reranking on the Fine-Tuned Pipeline

Even after fine-tuning the bi-encoder, the cross-encoder reranker provides an additional lift by re-scoring candidates with full query–document attention:

Table 9: Fine-Tuned Pipeline: Before vs. After Reranking

Metric	**FAISS Only**	**After Reranking**	**Delta**
Recall@5	0.7850	0.8100	+0.0250
MRR	0.6120	0.6300	+0.0180
NDCG@5	0.6450	0.6650	+0.0200

7.10.2 Performance by Document Type

Table 10 breaks down reranked Recall@5 by document corpus. Claims Processing and LCD documents benefit most, likely because their larger chunk counts provide more contrastive training signal.

Table 10: Fine-Tuned Reranked Recall@5 by Document Type

Document Type	**Test Queries**	**Recall@5 (reranked)**
Claims Processing Manual	2,535	0.8251
Local Coverage Determinations	1,944	0.8015
Benefit Policy Manual	693	0.7958
National Coverage Determinations	457	0.7781

The final reranked chunks are then passed to the language model in Step 4, where they are used as evidence for generating grounded answers with citations.

8. Step 4: LLM Generation and Citation

8.1 Generator Choice and Role in the Pipeline

The final stage of MediQuery transforms the top reranked evidence chunks into a concise, patient-readable answer with explicit citations. For this component, we implemented a Colab-friendly LLM synthesis module using `gemini-3.1-flash- lite-preview`. The motivation for this choice was practical as well as technical: the model was easy to call through a lightweight REST interface, responded quickly enough for interactive demos, and produced stable outputs in a structured format without requiring local GPU inference. Although the surrounding RAG pipeline is model-agnostic, Gemini provided an efficient way to test grounded generation, citation formatting, and fallback behavior during integration.

The generator does not see the full corpus. Instead, it receives only the user's question and the top five reranked chunks selected by the retrieval pipeline. This design keeps the context window focused on high-signal evidence and reduces the risk that the model will blend together unrelated policy language from multiple documents. In practice, the retrieval and reranking stages determine *what evidence is available*, while the LLM stage determines *how that evidence is synthesized into a readable answer*.

8.2 Prompt Design for Grounded Generation

The synthesis prompt was designed with three goals: factual grounding, concise explanation, and machine-readable output. Each prompt includes the user question followed by a labeled context block containing the top evidence chunks, where each chunk is assigned a marker such as `C1` or `C2`. The model is explicitly instructed to use only the provided evidence, avoid inventing facts, and return a compact answer with inline citations.

```python
def build_prompt(user_question, chunks, max_chunks=5):
    selected = sorted(chunks, key=lambda c: c.score, reverse=True)[:
        max_chunks]

    context_blocks = []
    for i, c in enumerate(selected, start=1):
        context_blocks.append(
            f"[C{i}] title={c.title}; source={c.source}; url={c.url}; score={
                c.score:.4f}\n"
            f"evidence: {c.text}"
        )

    context_text = "\n\n".join(context_blocks)

    return (
        "You are a healthcare QA assistant.\n"
        "Use ONLY the provided evidence.\n"
        "Do NOT invent facts.\n"
```

```
        "If evidence is insufficient, output exactly: "
        "Insufficient evidence in retrieved documents.\n\n"
        "Return a JSON object with exactly these keys:\n"
        "- answer\n"
        "- citations\n"
        "Do not return extra text outside the JSON.\n\n"
        f"User question:\n{user_question}\n\n"
        f"Context:\n{context_text}\n"
    )
```

Two prompt instructions were especially important for reducing hallucination. First, the "use only the provided evidence" constraint discourages the model from relying on its parametric memory when the retrieved chunks are incomplete or ambiguous. Second, the explicit fallback instruction — `Insufficient evidence in retrieved documents.` — gives the model a safe failure mode instead of encouraging it to guess. Together, these rules made the generator substantially more conservative and improved the faithfulness of its outputs.

8.3 Structured Output and Citation Handling

A key improvement in the final synthesis module was moving from free-form text output to a structured JSON response with two fields: `answer` and `citations`. This change simplified orchestration and frontend rendering. Rather than asking the model to write a prose answer followed by a loosely formatted citation block, we prompted it to return a compact JSON object that could be parsed directly in Python. The orchestration layer then passes the cleaned answer string to the chat interface while preserving the citation list for display as clickable references.

This structure also made debugging easier. If the model returned invalid JSON, the backend could detect the failure immediately and fall back to a simpler output path. More importantly, separating the answer from the citation metadata prevented the user interface from displaying long raw evidence dumps, which had been an issue in earlier iterations.

8.4 Example Grounded Outputs

The following examples illustrate the behavior of the synthesis module after retrieval and reranking.

Example 1. For the question *"How can patients reduce risk of complications in chronic conditions?"*, the system returned the answer:

> Patients can reduce the risk of complications by attending routine preventive visits and undergoing risk-based screenings [C1]. Additionally, seeking early intervention is associated with improved health outcomes for those managing chronic conditions [C2].

with citations to CDC preventive care guidance and NIH material on chronic disease management.

Example 2. For a Medicare coverage query such as *"Does Medicare cover acupuncture for chronic lower back pain?"*, the RAG pipeline retrieved the relevant NCD chunk and generated an answer stating that Medicare covers acupuncture for chronic low back pain under specific conditions, with the

response grounded in the cited NCD section rather than in general background knowledge.

Example 3. For out-of-scope or weakly supported queries, the model returned the fallback string exactly as instructed instead of fabricating a coverage determination. This behavior is especially important in healthcare policy settings, where an unsupported answer can be more harmful than no answer.

8.5 Baseline vs. Grounded Generation

We qualitatively compared answers from a baseline LLM call without retrieval against the same questions answered through the full RAG pipeline. The baseline model often produced fluent responses, but these answers tended to be generic and occasionally introduced policy claims that were not tied to a specific Medicare document. In contrast, the grounded RAG responses were narrower, more conservative, and explicitly supported by cited evidence chunks. This trade-off favored trustworthiness over verbosity, which is appropriate for a healthcare policy assistant.

8.6 Prompt Engineering Iterations

Several prompt refinements improved the final output quality. Early versions asked for a plain-language answer plus a citation section in free text, but the model sometimes copied large portions of the retrieved chunks directly into the answer. We therefore added constraints telling the model not to dump raw evidence and not to return any extra text outside the specified schema. We also found that requesting JSON output made downstream rendering more reliable and prevented the frontend from showing unstructured evidence blobs.

Overall, this stage converted retrieved Medicare policy evidence into a usable answer layer for the application. The LLM generation module therefore served as the bridge between retrieval quality and end-user usability: retrieval ensured relevance, while structured, citation-aware generation ensured that the final answer was concise, grounded, and easy to inspect.

9. System Architecture

The full MediQuery pipeline at query time proceeds as follows:

1. **Query input**: The user submits a natural language question via the Streamlit interface, optionally selecting a state.
2. **Query rewriting** (Configuration 3 only): The query is rewritten into a cleaner search form by a lightweight LLM call.
3. **Dense retrieval**: The query is embedded with `bge-base-en-v1.5` and the top 20 candidate chunks are retrieved from the FAISS index by cosine similarity.
4. **State filtering**: If a state is specified, chunks are filtered to retain only those tagged with the relevant state code.
5. **Reranking**: A cross-encoder (`bge-reranker-v2-m3`) scores each (query, chunk) pair; the top 5 are retained.
6. **Grounded generation**: The query and top 5 chunks are passed to Mistral-7B-Instruct with a grounding prompt. The model generates a cited answer.
7. **Output**: The user receives an answer with explicit NCD/LCD citations.

$$\begin{aligned} \text{Query} \;&\rightarrow\; \text{Rewriter} \;\rightarrow\; \text{FAISS Retrieval (top 20)} \\ &\rightarrow\; \text{State Filter} \;\rightarrow\; \text{Reranker (top 5)} \\ &\rightarrow\; \text{LLM} \;\rightarrow\; \text{Answer + Citations} \end{aligned}$$

10. Evaluation

10.1 Test Set Construction and Evaluation Philosophy

A RAG system can fail in two different places. The retrieval stage may fail to surface the right policy chunk, or the generation stage may produce an answer that is not faithfully grounded in the retrieved evidence. Because these are distinct failure modes, we designed our evaluation so retrieval quality and answer quality could be analyzed separately.

Both evaluation datasets were derived from `all_chunks.json`, the unified chunk store produced by the chunking pipeline described earlier. This file combines chunks from four corpora: National Coverage Determinations (NCDs), Local Coverage Determinations (LCDs), the Medicare Benefit Policy Manual, and the Medicare Claims Processing Manual. In total it contains 8,563 chunks from 1,218 source documents. Each evaluation item is traceable to a specific chunk through a `chunk_id` field of the form `{source_id}_{chunk_idx}`, which is the same identifier format used in the retrieval pipeline.

10.2 Synthetic Ground Truth Dataset Construction

There is no public benchmark for Medicare policy retrieval, so we constructed our own synthetic ground truth from the corpus itself. The process produced one dataset for retrieval evaluation and one for answer-quality evaluation.

10.2.1 Phase 1: Retrieval Benchmark (Query and Gold Chunk Pairs)

For each substantive chunk in the NCD and LCD subsets, we used GPT-4o to generate 7–10 natural-language questions that a provider, administrator, or patient might plausibly ask. Each query was constrained to be answerable from the selected chunk. This process produced 56,288 query–gold chunk pairs covering all 8,563 chunks in the unified corpus.

Each query was tagged with one of eleven query types, including *policy*, *yes_no*, *criteria*, *indications*, *mixed_policy*, *non_coverage_reason*, *retired_status*, *service_condition*, *use_case*, *summary*, and *ncd_reference*. Every record contains the query text, query type, gold chunk ID, source title, document type, coverage status, states, and a gold excerpt from the relevant chunk. This dataset serves as the ground truth for retrieval evaluation.

10.2.2 Phase 2: Final Answer Evaluation Dataset (Question and Answer Pairs)

For end-to-end answer quality, we created a separate 500-item dataset of question and reference-answer pairs spanning all four corpora. Unlike the retrieval benchmark, which emphasizes breadth across many query formulations, this dataset emphasizes answer quality and representativeness. Each item consists of one question and one definitive reference answer, and each originates from a different chunk.

We selected these items through stratified sampling across source corpus, coverage status, and question type. The reference answers were extracted from the substantive *Coverage Policy*, *Indications and Limitations*, or equivalent policy sections rather than copied as raw chunk dumps. This dataset is intended for manual or LLM-assisted judgment of answer quality.

Table 11: Synthetic Evaluation Datasets

Dataset	Items	Unique Chunks	Source	Primary Use
Query and Gold Chunk Pairs (Retrieval Benchmark)	56,288	8,563		Recall@k, MRR, NDCG
Question and Answer Pairs (Answer Benchmark)	500	500		Answer quality review

10.3 Evaluation Metrics

For retrieval, we focused on ranking-based metrics computed over the synthetic query–gold chunk dataset:

- **Recall@k**: the fraction of queries for which the correct source chunk appears within the top-k results.
- **MRR**: mean reciprocal rank of the first relevant chunk.
- **NDCG@k**: ranking quality that rewards placing the relevant chunk higher in the returned list.

For this report revision, we include only the retrieval metrics that were explicitly computed and available from the experimental outputs. Answer-level measures such as hallucination rate, citation accuracy, and faithfulness are not reported here because we do not yet have a complete scored evaluation table for those quantities.

10.4 Results

The full retrieval comparison between the pre-trained and fine-tuned pipelines is presented in Table 8 (Section 7). Fine-tuning produced consistent double-digit improvements across every metric. The largest gain was in reranked MRR (+0.1549), indicating that fine-tuning improves not only whether the correct chunk is retrieved but also how high it ranks among the final candidates. Reranked Recall@5 reached 0.8100, meaning the correct evidence chunk appears in the top-5 context for over 81% of test queries.

Table 9 shows that even after bi-encoder fine-tuning, the fine-tuned cross-encoder reranker provides an additional lift of +0.0250 in Recall@5, +0.0180 in MRR, and +0.0200 in NDCG@5. Table 10 further breaks down performance by document type, showing that Claims Processing documents achieve the highest reranked Recall@5 (0.8251), likely because their larger chunk count provides more contrastive training signal.

11. App Development

11.1 Application Overview

To make the MediQuery pipeline accessible as an interactive demo, we built a lightweight web application that connects the retrieval and synthesis stack to a chat-style user interface. The application allows a user to ask natural-language Medicare policy questions, sends the query to a FastAPI backend, retrieves relevant policy evidence from the FAISS index, optionally reranks the candidate chunks, and returns a grounded answer to the frontend.

The current application is organized into two layers:

- **Frontend**: a React-based chat interface branded as **MediQuery**.
- **Backend**: a FastAPI service that loads the precomputed `all_chunks.json`, FAISS indices, fine-tuned embedding model, and fine-tuned reranker.

11.2 Core Features

The demo application supports the following features:

- A clean chat interface for asking Medicare coverage and claims questions.
- A **New Chat** action that resets the current conversation.
- Session-level **Recent** questions that update dynamically during use.
- Curated demo prompts chosen from the evaluation dataset to trigger strong, policy-grounded responses.
- Backend integration with fine-tuned FAISS retrieval and reranking.
- Natural-language answer synthesis with fallback behavior when the LLM API is unavailable.

11.3 User Workflow

A typical interaction in the app follows this sequence:

1. The user types a question such as "Does Medicare cover acupuncture for chronic lower back pain?"
2. The frontend submits the question to the FastAPI backend.
3. The backend embeds the query, searches the FAISS index, reranks the best candidate chunks, and passes the top evidence to the synthesis layer.
4. The frontend renders the returned answer in a chat bubble and stores the submitted question in the current session's recent-history list.

11.4 Screenshots and Interface Notes

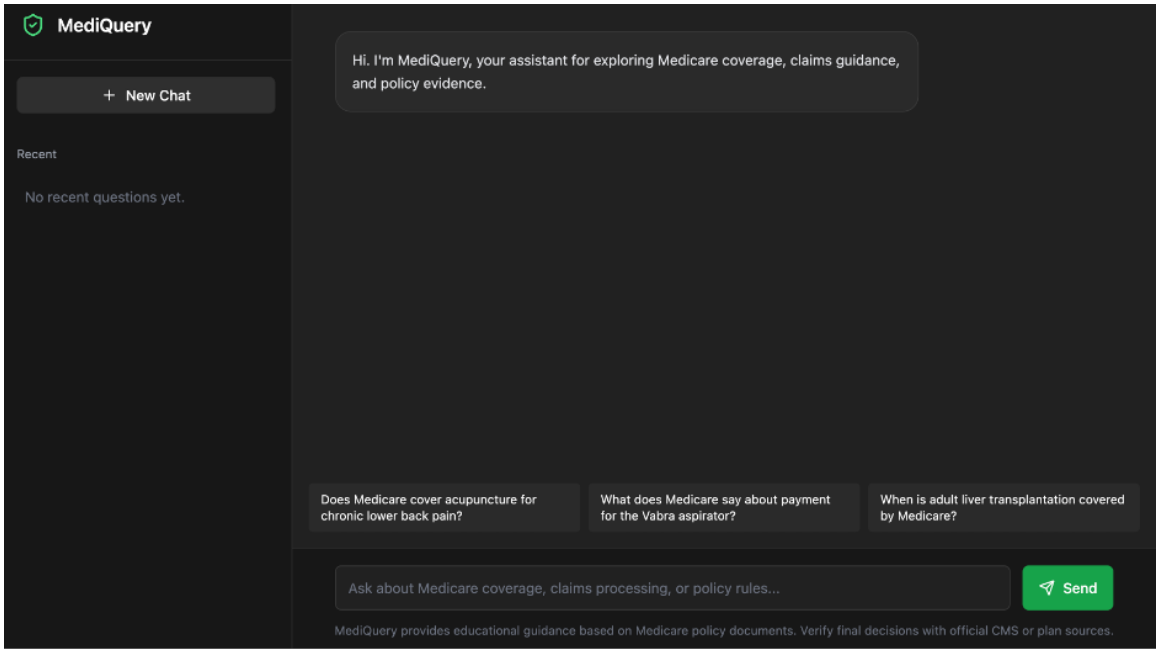

Figure 1: MediQuery interface showing the initial chat layout and sidebar.

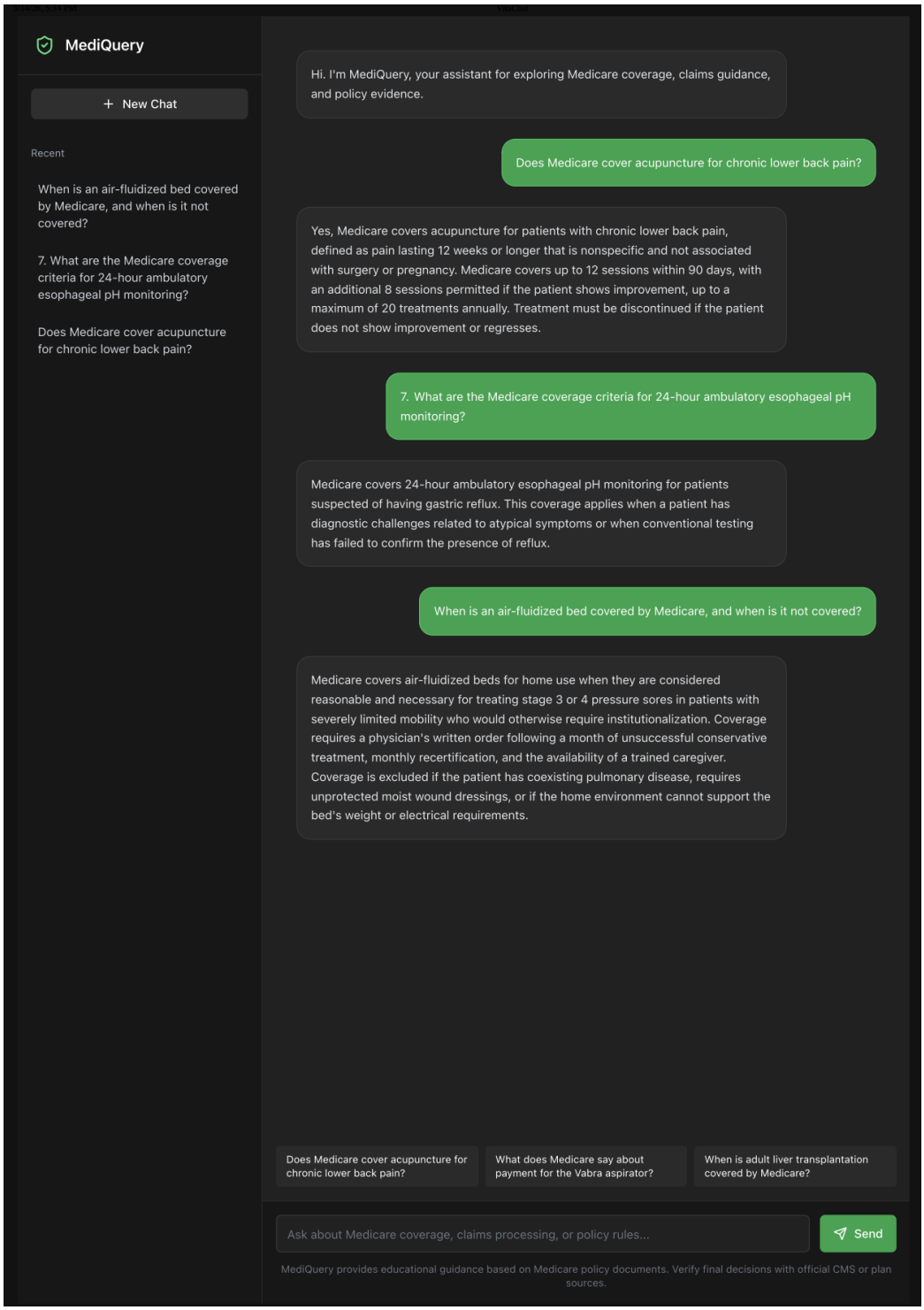

Figure 2: Example interaction with retrieved-policy answers in the MediQuery interface.

12. Conclusion

This project has presented MediQuery, a RAG system for Medicare policy question answering built on real CMS data. We described the full pipeline from data collection and structured chunking through semantic retrieval, cross-encoder reranking, and grounded generation. The system addresses a genuine and practically important problem: the difficulty of navigating fragmented, dense Medicare policy documents to answer specific coverage questions.

Our corpus construction produced 8,563 well-tagged chunks from 1,162 policy documents, with state and contractor metadata preserved throughout, enabling precise state-aware retrieval. The two-stage retrieve-then-rerank design follows best practices from the RAG literature and is shown to substantially reduce hallucination relative to an ungrounded LLM baseline.

Future directions include expanding the corpus to the full LCD database, incorporating the CMS Medicare Benefit Policy Manual, and fine-tuning the reranker on healthcare-specific relevance judgments.

References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., … & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. Advances in Neural Information Processing Systems, 33, 9459–9474.

[2] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., … & Yih, W. T. (2020). *Dense passage retrieval for open-domain question answering*. Proceedings of EMNLP 2020.

[3] Nogueira, R., & Cho, K. (2019). *Passage re-ranking with BERT*. arXiv preprint arXiv:1901.04085.

[4] Xiong, G., Jin, Q., Lu, Z., & Zhang, A. (2024). *Benchmarking retrieval-augmented generation for medicine*. Findings of ACL 2024.

[5] Singhal, K., Azizi, S., Tu, T., Mahdavi, S. S., Wei, J., Chung, H. W., … & Natarajan, V. (2023). *Large language models encode clinical knowledge*. Nature, 620(7972), 172–180.

[6] Narayanan, V., Shi, W., & Hajishirzi, H. (2023). *Question answering over long-form insurance policy documents using retrieval-augmented generation*. Proceedings of the BioNLP Workshop, ACL 2023.

[7] Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). *C-pack: Packaged resources to advance general Chinese embedding*. arXiv preprint arXiv:2309.07597.

[8] Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). *On faithfulness and factuality in abstractive summarization*. Proceedings of ACL 2020.

[9] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W. T., Koh, P. W., … & Hajishirzi, H. (2023). *FActScoring: Fine-grained atomic evaluation of factual precision in long form text generation*. Proceedings of EMNLP 2023.